R4DS 15 – Quarto

Quarto Basics

What is Quarto?

Quarto is a unified authoring framework for data science:

- Combine **code**, **results**, and **prose** in one document
- Supports R, Python, Julia, and Observable JS
- A command-line tool (not an R package)

Three uses:

1. **Communicate** to decision-makers (hide code, show results)
2. **Collaborate** with other data scientists (show code + reasoning)
3. **Environment** for doing data science (notebook for yourself)

Pipeline: `.qmd` → **knitr** (executes R) → `.md` → **Pandoc** (converts) → output

Document Structure

A `.qmd` file has three types of content:

1. **YAML header** (between `---`):

```yaml
title: "Diamond Sizes"
author: "Jane Doe"
date: today
format: html
```

2. **Code chunks** (between ```` ``` ````):

```r
smaller <- diamonds |> filter(carat <= 2.5)
```

3. **Text** with Markdown formatting: `**bold**`, `*italic*`, `` `code` ``, `# Heading`, `- bullet`, `[link](url)`.

Code Chunks and Labels

Insert chunks with:

- **Keyboard:** Ctrl+Alt+I (Mac: Cmd+Option+I)
- **Button:** Insert → Code Chunk
- **Manual:** type `` ```{r} `` and `` ``` ``

Labels identify chunks for navigation and cross-references:

```
1 + 1
```

Label rules: one word or words connected by **dashes**. Avoid `.`, `_`, and spaces. Figures must start with `fig-`, tables with `tbl-`.

Key options (set with #| inside chunks):

Option	Effect
`eval: false`	Show code, don't run it
`include: false`	Run code, hide everything
`echo: false`	Run code, hide the code
`results: hide`	Run code, hide text output
`fig-show: hide`	Run code, hide plots
`message: false`	Suppress messages
`warning: false`	Suppress warnings

Global Options and Inline Code

Global options apply to all chunks via YAML:

```yaml
title: "My Report"
execute:
  echo: false
  warning: false
```

Override per-chunk with `#| echo: true`.

Inline code embeds R results in text:

```
We have data about 53940 diamonds.
```

Useful helper for readable numbers:

```r
comma <- function(x) format(x, digits = 2,
                            big.mark = ",")
comma(3452345)
```

```
#> [1] "3,452,345"
```

Exercises (Basics)

1. Create a new `.qmd` file. Render to HTML, PDF, and Word. How do the outputs differ?
2. Create a brief CV in Quarto: your name as title, headings for education and employment, bulleted lists, bold years.
3. Set `echo: false` globally. Write a report on diamond sizes for a non-R audience.
4. Use `scales::label_comma()` for readable inline numbers. Compute the percentage of diamonds > 2.5 carats.

Solutions (Basics)

1. HTML is interactive (scrollable). PDF is fixed-layout (good for print). Word is editable. The `.qmd` input is identical — only `format:` changes.
2. Use `**2020**` for bold years in a bulleted list under `## Education` and `## Employment`.
3. Set in YAML: `execute: echo: false`. Use `ggplot()` for visualizations, `knitr::kable()` for tables.
4.

```r
# Inline: `r scales::label_comma()(nrow(diamonds))`
# Percentage:
pct <- round(sum(diamonds$carat > 2.5) /
  nrow(diamonds) * 100, 1)
# Inline: `r pct`% are larger than 2.5 carats
```

Figures, Tables, and Caching

Control with chunk options:

Option	Effect
`fig-width`	Width in inches (default 7)
`fig-height`	Height in inches
`fig-asp`	Aspect ratio (height/width)
`out-width`	Scaling in output (e.g., "70%")
`layout-ncol`	Multi-figure layout

Recommendations:

- Set `fig-width` between 4–8 inches
- Use `fig-asp: 0.618` (golden ratio) for aesthetics
- `out-width: "70%"` for a good default size in documents
- Cross-reference figures with `fig-` label prefix + `fig-cap`

Tables with `knitr::kable()`

Default prints are basic. Use `knitr::kable()` for polished tables:

```
knitr::kable(mtcars[1:4, 1:4])
```

	mpg	cyl	disp	hp
Mazda RX4	21.0	6	160	110
Mazda RX4 Wag	21.0	6	160	110
Datsun 710	22.8	4	108	93
Hornet 4 Drive	21.4	6	258	110

Cross-reference with `tbl-` label prefix and `tbl-cap`:

Table 1: First 5 cars

```
knitr::kable(mtcars[1:5, ])
```

Other packages: `gt`, `kableExtra`, `huxtable`, `flextable`.

Expensive computations can be cached:

```
rawdata <- read_csv("a_very_large_file.csv")
```

Track dependencies with `dependson`:

```
processed <- rawdata |>
  filter(!is.na(important_var))
```

Invalidate when source file changes:

```
cache.extra: !expr file.mtime("large_file.csv")
```

Or enable document-wide: `execute: cache: true` in YAML.

Exercises (Figures/Tables/Caching)

1. Try each figure option one at a time: `fig-width: 10`, `fig-height: 3`, `out-width: "100%"`, `out-width: "20%"`. How does each change the output?
2. Display `diamonds[1:5, ]` with `knitr::kable()`. Add a `tbl-` label and `tbl-cap`. Cross-reference it in text.
3. Set up cached chunks: `d` depends on `c` and `b`, both depend on `a`. Print `lubridate::now()` in each. Verify caching behavior.

1. `fig-width: 10`: wider figure, elements appear smaller. `fig-height: 3`: short, wide figure. `out-width: "100%"`: fills page width. `out-width: "20%"`: tiny figure.
2.

Table 2: First 5 diamonds

```r
knitr::kable(diamonds[1:5, ])
```

Reference in text: `@tbl-diamonds`.

3. Use `dependson: "a"` for b and c, `dependson: c("b", "c")` for d. Only a re-runs when its code changes; downstream chunks cascade.

Configuration

Self-contained HTML embeds all images, CSS, JS:

```yaml
format:
  html:
    embed-resources: true
```

Parameters make documents reusable:

```yaml
params:
  start: !expr lubridate::ymd("2015-01-01")
  snapshot: !expr lubridate::ymd_hms("2015-01-01 12:30:00")
```

Access with `params$start` in R code. Render with different values programmatically.

Add a `.bib` file and CSL style:

```
bibliography: references.bib
csl: apa.csl
```

Citation syntax in text:

Syntax	Output
`[@smith04]`	(Smith 2004)
`@smith04`	Smith (2004)
`[@smith04; @doe99]`	(Smith 2004; Doe 1999)
`[-@smith04]`	(2004) — suppress author

Bibliography is auto-inserted at document end.

Troubleshooting

Common issues and fixes:

1. **Duplicated chunk labels** — every label must be unique
2. **Working directory** — Quarto renders from the `.qmd` file's directory
3. **Object not found** — code runs top-to-bottom during render; test with "Restart R and Run All Chunks"
4. **Hard-to-debug errors** — add `error: true` to show the error in output
5. **Package conflicts** — use `pkg::fun()` calls or `conflicted` package

Output Formats

Set output in YAML:

Format	YAML	Output
HTML	`html`	Web page
PDF	`pdf`	Fixed-layout (needs LaTeX)
Word	`docx`	Editable document
Markdown	`gfm`	GitHub-flavored markdown

Multiple formats from one file:

```yaml
format:
  html:
    toc: true
  pdf: default
  docx: default
```

Render all: `quarto render file.qmd --to all`

Quarto creates slides with header-based divisions:

Format	YAML	Technology
RevealJS	`revealjs`	HTML/JS
PowerPoint	`pptx`	MS Office
Beamer	`beamer`	LaTeX/PDF

- # creates section title slides
- ## creates content slides
- --- creates untitled slides
- Speaker notes go in `:::{.notes}` divs

These slides are built with Quarto + Beamer!

htmlwidgets — client-side (no R server needed):

```r
library(leaflet)
leaflet() |>
  setView(174.764, -36.877, zoom = 16) |>
  addTiles() |>
  addMarkers(174.764, -36.877,
    popup = "Maungawhau")
```

Packages: `leaflet` (maps), `DT` (tables), `plotly` (interactive plots).

Shiny — server-side (needs R running): add `server: shiny` to YAML.

Websites and Books

Multi-document projects use `_quarto.yml`:

```yaml
project:
  type: website

website:
  title: "My Site"
  navbar:
    left:
      - href: index.qmd
        text: Home
      - about.qmd
```

- **Websites:** linked pages with navigation bars
- **Books:** ordered chapters; supports HTML + PDF + EPUB
- Render all with `quarto render` from project root